Dyer
Innovation

CLAUDE CERTIFIED ARCHITECT · SCENARIO 1

Customer Support Resolution Agent

A study guide for the intervention hierarchy, valid escalation triggers, structured error and handoff patterns, and context management for long support conversations.

LAST UPDATED

April 27, 2026

THIS GUIDE EMPHASIZES

Intervention hierarchy (descriptions to architecture) · Valid vs invalid escalation triggers · Structured error responses for MCP tools · Case facts block for context preservation

Contents

01 Scenario Overview 3

02 The Intervention Hierarchy 4

2.1 Key Distinction: Skill Gap vs Safety Constraint 4

03 Few-Shot Examples vs Architectural Decomposition 5

04 Valid Escalation Triggers: Policy Gaps vs Discomfort 6

4.1 Invalid Escalation Triggers (Exam Traps) 6

05 The Evaluator-Optimizer Pattern 7

5.1 When To Use Each Approach 7

06 Structured Error Responses for MCP Tools (Domain 2, Task 2.2) 8

6.1 The Structured Error Response Pattern 8

6.2 Error Categories and Agent Behavior 8

6.3 Applied Example: process_refund Error Response 8

6.4 Anti-Pattern: Access Failure vs Valid Empty Results 8

07 Structured Handoff Summaries (Domain 1, Task 1.4) 10

7.1 Why Handoff Summaries Matter 10

7.2 The Required Handoff Fields 10

7.3 Applied Example 10

01 Scenario Overview

The scenario at a glance:

- Scenario 1 of the Anthropic Claude Certified Architect exam centers on a customer support resolution agent built with the Claude Agent SDK
- The agent handles high-ambiguity requests like returns, billing disputes, and account issues
- It has access to backend systems through custom MCP tools: `get_customer`, `lookup_order`, `process_refund`, and `escalate_to_human`
- Target: 80%+ first-contact resolution while knowing when to escalate

Domains covered:

- **Domain 1 (Agentic Architecture & Orchestration)** — dominant
- **Domain 5 (Context Management & Reliability)** — dominant
- **Domain 4 (Prompt Engineering)** — touchpoints around tool selection and escalation framing
- **Domain 2 (Tool Design & MCP Integration)** — supporting material for structured error responses and tool description quality

Core skill the exam evaluates: pick the right level of intervention. Not every problem needs an architectural change; not every problem can be fixed with a better prompt. Knowing which is which is the central decision repeatedly tested.

02 The Intervention Hierarchy

The exam consistently tests whether candidates can select the right level of intervention. There is an implicit hierarchy the exam expects:

LEVEL	INTERVENTION	USE WHEN	EXAMPLE
1	Tool descriptions	Root cause is tools are indistinguishable	Minimal descriptions to expand with input formats, edge cases, boundaries
2	Few-shot examples	Agent has the capability but lacks pattern guidance	94% on singles, 58% on multi-concern; show the multi-concern pattern
3	System prompt rules	Behavior needs explicit boundaries or criteria	Escalation miscalibration to add explicit criteria
4	Programmatic hooks	Hard business rule that must NEVER be violated	Identity verification before refunds: a prerequisite gate
5	Architectural changes	Simpler fixes tried and failed, or fundamental capability gap	Preprocessing layer, separate classifier, model tier upgrade

RULE

Exam Decision Rule

- Always ask: has the simplest proportionate fix been tried?
- The question will tell you if lower-level fixes have already been attempted
- If not, start at the lowest applicable level
- Only escalate to architectural solutions when:
  - The problem requires enforcement guarantees, OR
  - Simpler fixes are explicitly ruled out

Key Distinction: Skill Gap vs Safety Constraint

The fundamental fork in the exam's decision tree:

- **Skill gap** = the agent doesn't know the right pattern
- **Safety constraint** = the agent must be prevented from violating a rule
- Each demands a different category of solution

SKILL GAP (PROMPT-LEVEL FIX)	SAFETY CONSTRAINT (PROGRAMMATIC FIX)
Agent can do the task but doesn't know when/how	Agent must NEVER skip a step or violate a rule
Errors are about quality/consistency	Errors cause financial/legal/safety harm
Few-shot examples, system prompt criteria	PostToolUse hooks, prerequisite gates
Multi-concern handling, tool selection, escalation calibration	Identity verification before refunds, blocking high-value operations

03 Few-Shot Examples vs Architectural Decomposition

Common exam pattern:

- Agent handles individual concerns at high accuracy (e.g., 94%)
- Agent underperforms on multi-concern messages (e.g., 58% tool-selection accuracy)
- Intuitive but wrong move: preprocessing layer with a separate model call to decompose the message
- That is over-engineering for this failure mode

EXAMPLE

Why few-shot examples are the right fix:

- The agent has the underlying capability; it just lacks pattern guidance for the multi-concern case
- **Few-shot examples are the proportionate, low-cost fix** — they teach the model a decomposition pattern it can generalize to novel multi-concern combinations
- A preprocessing layer is appropriate only when:
  - Simpler fixes have been tried and failed, OR
  - The agent fundamentally cannot decompose without external help

04 Valid Escalation Triggers: Policy Gaps vs Discomfort

The exam guide defines a precise set of valid escalation triggers. Memorize these — they appear repeatedly:

VALID TRIGGER	EXAMPLE	WHY IT'S VALID
Customer explicitly requests a human	"I want to speak to a manager"	Customer autonomy must be honored immediately
Policy gap or ambiguity	Competitor price matching when policy only covers own-site adjustments	Agent has no authority to interpret policy
Agent cannot make meaningful progress	All troubleshooting steps exhausted, issue persists	Further attempts waste customer time

Invalid Escalation Triggers (Exam Traps)

The exam frequently offers these as distractors. Each one sounds reasonable but fails the validity test:

INVALID TRIGGER	WHY IT'S WRONG	WHAT TO DO INSTEAD
Contradictory evidence might upset the customer	Discomfort is not a policy gap; the agent CAN handle delivery disputes	Present evidence diplomatically, offer next steps
Self-reported confidence score is low	LLM confidence scores are poorly calibrated and unreliable	Use explicit criteria, not confidence numbers
Customer sentiment is negative	Sentiment doesn't correlate with case complexity	Acknowledge frustration, attempt resolution
The request has multiple parts	Multi-concern requests are within agent capability	Decompose and handle each concern
Customer might change their mind	Speculating about future feelings is not actionable	Process the stated request per policy

Exam Pattern

- When a question asks about escalation, check if the scenario describes a POLICY GAP (policy is silent or ambiguous on the specific request)
- If yes, that's almost always the correct escalation trigger
- If the other options describe any of the following, they're distractors:
    - Emotional discomfort
    - Prediction of customer behavior
    - Self-assessed confidence

05 The Evaluator-Optimizer Pattern

The evaluator-optimizer pattern adds a self-critique step before the agent presents its response. The flow:

- Agent generates a draft
- Agent evaluates that draft against specific completeness criteria (e.g., does it include policy context, timelines, and next steps?)
- Agent revises as needed before sending

When To Use Each Approach

FEW-SHOT EXAMPLES	EVALUATOR-OPTIMIZER PATTERN
Gaps are predictable and follow known patterns	Gaps are unpredictable and vary case-by-case
You can enumerate the common case types	The combination of missing elements is different each time
Teaches "what good looks like" for known scenarios	Checks "whether this specific response is good" regardless of scenario
Static: same examples for every request	Dynamic: adapts criteria evaluation to each individual response
Best for: format consistency, tool selection patterns, escalation calibration	Best for: response completeness, quality assurance, catching case-specific omissions

Key Signal in the Question

- The phrase "specific context gaps vary by case" is the decisive clue
- When variability is the problem, static examples can't cover it
- You need a dynamic quality check — that's the evaluator-optimizer pattern
- Mental model:
  - Few-shot examples = a CHECKLIST of known patterns
  - Evaluator-optimizer = a REVIEWER who reads each response fresh

06 Structured Error Responses for MCP Tools (Domain 2, Task 2.2)

Why generic errors fail the agent:

- When an MCP tool returns "Operation failed" or "Error occurred," the agent cannot make an intelligent recovery decision
- It cannot distinguish between:
  - A temporary network timeout (worth retrying)
  - An invalid input (needs correction)
  - A policy violation (should explain to customer)
  - A permission issue (needs escalation)
- Generic errors force the agent into a one-size-fits-all failure response

The Structured Error Response Pattern

Every MCP tool should return structured error metadata that enables the agent to take the right next action. The standard pattern includes three key fields:

FIELD	PURPOSE	EXAMPLE VALUES
`errorCategory`	Tells the agent what KIND of failure occurred	`"transient"`, `"validation"`, `"business"`, `"permission"`
`isRetryable`	Tells the agent whether retrying could succeed	`true` (for timeouts), `false` (for policy violations)
`description`	Human-readable message the agent can relay or use for reasoning	"Refund amount exceeds $500 policy limit. Requires manager approval."

Error Categories and Agent Behavior

Each error category maps to a distinct agent response pattern:

CATEGORY	EXAMPLE SCENARIO	ISRETRYABLE	EXPECTED AGENT BEHAVIOR
Transient	Database timeout, service temporarily unavailable	`true`	Retry automatically (up to a limit), then explain delay to customer
Validation	Invalid order number format, missing required field	`false`	Ask customer for corrected information
Business	Refund exceeds $500 limit, return window expired	`false`	Explain the policy to the customer with a friendly description
Permission	Agent lacks authorization for this operation	`false`	Escalate to human agent with structured handoff

Applied Example: process_refund Error Response

When the agent calls `process_refund` for a $750 return:

- **Generic (bad) response:** "Refund failed"
- **Structured (good) response:**
  - `errorCategory: "business"`
  - `isRetryable: false`
  - `description: "Refund amount of $750 exceeds the $500 automatic approval limit. This r`
- **Resulting customer message (good):** "I can see your refund qualifies, but because it's above our autom to a manager for authorization. Let me transfer you now."
- **Resulting customer message (bad, with generic error):** "I'm sorry, I wasn't able to process your refund help."

Anti-Pattern: Access Failure vs Valid Empty Results

Critical distinction the exam tests:

- A tool that times out (access failure) is fundamentally different from a tool that successfully queries but finds no matching records (valid empty result)

07 Structured Handoff Summaries (Domain 1, Task 1.4)

The exam tests not only WHEN to escalate but WHAT to include when escalating. The exam guide requires a specific structured handoff pattern.

Why Handoff Summaries Matter

- When the agent escalates to a human, the human agent typically does NOT have access to the full conversation transcript
- They receive a queue ticket
- Without a structured summary, the human agent must re-ask the customer every question the AI agent already answered
- Result: terrible customer experience

The Required Handoff Fields

FIELD	WHAT IT CONTAINS	WHY IT'S NEEDED
Customer ID	Verified customer identifier from `get_customer`	Human agent can pull up the account immediately
Root Cause	What the agent determined is the underlying issue	Human agent doesn't have to re-diagnose
Refund Amount / Key Data	Specific figures, order numbers, dates relevant to resolution	Human agent has the facts without re-asking
Recommended Action	What the agent believes should happen next	Gives the human agent a starting point for resolution

Applied Example

Bad escalation: "Customer needs help with a refund."

Good escalation (compiled handoff):

- **Customer ID:** CUST-48291
- **Root Cause:** Customer was charged twice for Order #7834 ($129.99 each) due to a payment processing error on March 3. One charge is confirmed duplicate.
- **Recommended Action:** Process refund of $129.99 for the duplicate charge
- **Customer Communication:** Customer has been informed the refund will take 5–7 business days

This lets the human agent resolve the case in one interaction instead of starting from scratch.

08 Outbound Tool Call Interception (Domain 1, Task 1.5)

The exam distinguishes between PostToolUse hooks (which intercept results AFTER a tool executes) and the complementary pattern: intercepting outgoing tool calls BEFORE execution to enforce policy.

Two Directions of Hook Interception

POSTTOOLUSE (RESULTS INTERCEPTION)	PRE-CALL INTERCEPTION (OUTBOUND)
Fires AFTER the tool executes	Fires BEFORE the tool executes
Transforms/normalizes data the agent receives	Blocks or redirects tool calls that violate policy
Example: convert Unix timestamps to readable dates	Example: block `process_refund` when amount > $500

Applied Example: Refund Threshold Enforcement

- Business policy: refunds over $500 require human approval
- Agent calls `process_refund` with `amount: $750`
- A pre-call interception hook catches this before the refund executes, blocks the call, and redirects to the escalation workflow
- The agent never successfully calls `process_refund` for the $750
- The hook guarantees policy enforcement regardless of what the model decides
- **Distinction from prerequisite gate:**
  - Prerequisite gate = ensures steps happen in order
  - Pre-call interception = parameter-based policy check (specific values don't violate business rules)

Exam Distinction: Three Programmatic Mechanisms

- **Prerequisite gate:** Ensures Step A completes before Step B can start (identity verification before refund)

- **PostToolUse hook:** Transforms tool results after execution (data format normalization)
- **Pre-call interception:** Blocks or redirects tool calls before execution based on parameters (refund amount exceeds policy limit)
- All three are programmatic enforcement — the exam may test whether you can pick the right one for a given scenario

09 Trimming Verbose Tool Outputs (Domain 5, Task 5.1)

The problem:

- `lookup_order` may return a full order record with 40+ fields: internal IDs, warehouse codes, shipping carrier details, tax calculations, audit timestamps, etc.
- The agent only needs ~5 fields for the current task (order status, item name, amount, date, return eligibility)
- If verbose results accumulate across multiple tool calls in a long conversation, they consume a disproportionate share of the context window
- Result: actual conversation content gets crowded out

The Solution: Trim Before Accumulation

- Use a PostToolUse hook or result filtering step to trim tool outputs to only the fields relevant to the agent's task before they enter the conversation context
- For a return-related query:
  - **Keep:** `order_id`, `status`, `item_name`, `amount`, `purchase_date`, `return_eligible`
  - **Drop:** `internal_warehouse_id`, `tax_breakdown`, `shipping_tracking_events`, `audit_log`, etc.

Relationship to Case Facts Block

These two patterns work together to manage both ends of the context management problem:

- **Trimming verbose outputs** → prevents context bloat from tool results (too much irrelevant data coming in)
- **Case facts block** → prevents critical data loss from summarization (too little critical data being preserved)

10 Position-Aware Context Organization (Domain 5, Task 5.1)

The "lost in the middle" effect:

- Models reliably process information at the beginning and end of long inputs
- Models may miss or underweight findings from middle sections
- In a long customer support conversation or large aggregated input, critical details buried in the middle are more likely to be overlooked than details at the start or end

The Mitigation

- Place key findings summaries at the BEGINNING of aggregated inputs
- Organize detailed results with explicit section headers so the model can navigate rather than scanning linearly
- When passing results from multiple tool calls or subagents, lead with a summary of key facts before presenting the full details
- This puts the most important information in the position where it's most reliably processed

11 Handling Frustrated Customers (Domain 5, Task 5.2)

Scenario: customer is visibly frustrated (angry language, expressing dissatisfaction) but their actual issue is something the agent can resolve (standard return, billing correction, status update).

Correct pattern:

- **Step 1:** Acknowledge the frustration explicitly ("I understand this is frustrating")
- **Step 2:** Offer to resolve the issue ("Let me look into this and get it fixed for you right now")
- **Step 3:** Only escalate if the customer reiterates their preference for a human AFTER you've offered to help

Key insights:

- Frustration alone is NOT an escalation trigger
- The agent should attempt resolution first
- Many frustrated customers are satisfied when their issue gets resolved quickly, regardless of whether a human or AI does it
- But if the customer explicitly says "no, I want a person," honor that immediately — do not attempt to convince them otherwise

12 Key Patterns Reference

12.1 Agentic Loop Control (Domain 1, Task 1.1)

Core Mechanic: The `stop_reason` field in Claude's API response is the canonical control signal for agentic loops.

STOP_REASON	ACTION	EXPLANATION
`"tool_use"`	Continue the loop	Execute the requested tool(s), append results, call Claude again
`"end_turn"`	Stop the loop	Present Claude's text response to the customer

TRAP

Anti-Patterns (Common Exam Distractors)

- Parsing natural language signals ("I've completed") to detect loop termination
- Checking for assistant text content as a completion indicator (Claude can return text AND `tool_use` in the same turn)
- Using an arbitrary iteration cap as the PRIMARY stopping mechanism (acceptable as a safety fallback, not the main control)

12.2 Programmatic Enforcement vs Prompt Guidance (Domain 1, Tasks 1.4 & 1.5)

One of the most frequently tested distinctions in the entire exam.

RULE

Decision rule:

— Consequence of a miss = real-world harm (financial, legal, safety) → **programmatic enforcement**

— Consequence of a miss = quality degradation → **prompt-level guidance**

Programmatic Enforcement Mechanisms

- **Prerequisite gates:** Block downstream tool calls (e.g., `process_refund`) until prerequisite steps complete (e.g., `get_customer` returns a verified ID)
- **PostToolUse hooks:** Intercept tool results for data normalization before the agent processes them (e.g., converting Unix timestamps to readable dates)
- **Tool call interception:** Block policy-violating actions (e.g., refunds exceeding $500) and redirect to escalation workflows

Prompt-Level Guidance Mechanisms

- **Few-shot examples:** Teach patterns for ambiguous cases (tool selection, multi-concern handling, escalation calibration)
- **Explicit criteria in system prompt:** Define escalation boundaries, review criteria, severity levels
- **Evaluator-optimizer pattern:** Self-critique step for response quality assurance

12.3 Tool Description Design (Domain 2, Task 2.1)

RULE

Core Principle:

Tool descriptions are the primary mechanism LLMs use for tool selection. When tool selection is unreliable, fix descriptions first.

What Good Tool Descriptions Include

- **Purpose:** What the tool does and when to use it
- **Input formats:** What identifiers and parameters it accepts
- **Example queries:** Representative use cases
- **Edge cases:** Ambiguous situations and how they should be routed
- **Boundaries:** When NOT to use this tool and which tool to use instead

System Prompt Keyword Interference

- Keywords in system prompts can override well-written tool descriptions
- Example: if the system prompt says "when the customer mentions their account, verify their identity first," the word "account" can trigger `get_customer` even when `lookup_order` is more appropriate
- When a consistent keyword-triggered misrouting pattern persists despite good tool descriptions, examine the system prompt for keyword-sensitive instructions

12.4 Few-Shot Prompting Best Practices (Domain 4, Task 4.2)

RULE

Design Principles for Few-Shot Examples

— Target ambiguous scenarios, not clear-cut ones (the agent already handles easy cases)

— Use 2–6 targeted examples, not 10–15 broad ones (quality over quantity)

— Show reasoning for why one choice was made over plausible alternatives

— Enable generalization to novel patterns, not just pattern matching against examples

EXAMPLE

When Few-Shot Examples Are the Right Answer

— The agent has the underlying capability but lacks pattern guidance for a specific situation

— The failure mode involves inconsistency or ambiguity in judgment, not a hard safety constraint

— The question describes a "skill gap" not a "safety constraint"

— Simpler fixes (tool descriptions) have already been addressed or aren't the root cause

12.5 Context Management Patterns (Domain 5, Task 5.1)

Progressive Summarization Risks

- When context reaches capacity, older turns get summarized
- Specific transactional facts (amounts, dates, order numbers, percentages) get condensed into vague summaries like "discussed promotional pricing"

The Solution: Structured Case Facts Block

- Extract transactional facts into a persistent block that lives outside the summarized history
- This creates two separate layers of context (see table below)

SUMMARIZED HISTORY	CASE FACTS BLOCK (PERSISTENT)
Captures conversation flow and narrative	Stores specific amounts, dates, order numbers, statuses
Subject to compression as context grows	Always present in full, never summarized
"Customer discussed billing concern and promotional pricing"	"Discount: 15%, Order: #1234, Date: March 1, Amount: $89.99"

12.6 Escalation Patterns (Domain 5, Task 5.2)

The full escalation decision framework, combining valid triggers with handling patterns:

- Customer explicitly requests human → Honor immediately without attempting investigation
- Policy is ambiguous or silent → Escalate for policy interpretation
- Agent cannot make meaningful progress → Escalate with structured handoff summary
- Customer is frustrated but issue is within capability → Acknowledge frustration, attempt resolution; escalate only if customer reiterates preference
- Multiple customer matches from tool results → Ask for additional identifiers (email, phone, order number), never select based on heuristics

12.7 Tool Call Optimization (Domain 1)

Parallel Tool Requests

- Claude can request multiple tools in a single turn
- If the agent needs both customer info and order info, prompt it to batch both requests in one turn
- Orchestration code should execute all requested tools and return all results together before the next API call
- Benefit: reduces unnecessary round-trips

Multi-Concern Decomposition

For complex requests with multiple concerns (e.g., billing dispute + shipping update + cancellation), the agent should:

- Decompose the request into distinct concerns
- Fetch shared context once (customer ID, account info)
- Investigate each concern using that shared context
- Synthesize a unified resolution
- Benefit: avoids redundant data gathering

13 Core Terminology & Technology

TERM	DEFINITION
Claude Agent SDK	Framework for building agentic applications with Claude. Manages agent definitions, agentic loops, hooks, and subagent spawning.
MCP (Model Context Protocol)	Protocol for connecting Claude to external tools and data sources. MCP servers expose tools the agent can call.
Agentic Loop	The control flow pattern where orchestration code sends requests to Claude, checks `stop_reason`, executes tools if needed, and loops until `end_turn`.
`stop_reason`	Field in Claude's API response. `"tool_use"` = continue loop; `"end_turn"` = present response to user.
PostToolUse Hook	A programmatic hook that intercepts tool results after execution but before the agent processes them. Used for data normalization and format transformation.
Tool Call Interception	A hook that intercepts outgoing tool calls before execution. Used to enforce business rules like blocking high-value refunds.
Prerequisite Gate	Programmatic check that blocks downstream tool calls until prerequisite steps complete (e.g., verified customer ID required before refund).
Few-Shot Examples	2–6 targeted examples in the system prompt that demonstrate correct reasoning for ambiguous scenarios, enabling the model to generalize.
Evaluator-Optimizer Pattern	A self-critique step where the agent evaluates its own draft response against completeness criteria before presenting it. Catches case-specific quality gaps that vary across scenarios.
Progressive Summarization	Context management technique that summarizes older conversation turns when context capacity is reached. Risk: loses specific transactional facts.
Case Facts Block	A persistent structured block (amounts, dates, order numbers) included in each prompt, outside summarized history. Prevents fact loss during summarization.

14 Exam Domain Mapping

The following maps each concept in this guide back to its exam domain and task statement for targeted study.

Domain 1: Agentic Architecture & Orchestration (27%)

- Task 1.1: Agentic loop control via `stop_reason`
- Task 1.4: Programmatic prerequisites for workflow ordering
- Task 1.4: Structured handoff summaries for escalation
- Task 1.5: PostToolUse hooks for data normalization
- Task 1.5: Pre-call interception hooks for policy enforcement
- Task 1.6: Multi-concern decomposition with shared context
- Parallel tool batching to reduce round-trips

Domain 2: Tool Design & MCP Integration (18%)

- Task 2.1: Tool descriptions as primary selection mechanism
- Task 2.1: System prompt keyword interference on tool selection
- Task 2.2: Structured error responses — `errorCategory`, `isRetryable`, `isError` flag

Domain 4: Prompt Engineering & Structured Output (20%)

- Task 4.2: Few-shot examples for ambiguous tool selection — 2–6 targeted, with reasoning
- Task 4.2: Few-shot examples for multi-concern pattern guidance
- Task 4.2: Few-shot examples for escalation calibration
- Task 4.4: Evaluator-optimizer / self-critique pattern for response quality

Domain 5: Context Management & Reliability (15%)

- Task 5.1: Case facts extraction to prevent summarization loss
- Task 5.1: Trimming verbose tool outputs to relevant fields
- Task 5.1: Lost-in-the-middle mitigation with position-aware ordering
- Task 5.2: Valid escalation triggers — policy gaps, customer requests, inability to progress
- Task 5.2: Multiple customer matches → clarification, not heuristic selection
- Task 5.2: Unreliable proxies — self-reported confidence scores, sentiment analysis
- Task 5.2: Frustrated-but-resolvable customer handling pattern

15 Pre-Exam Study Checklist

Before taking the exam, confirm you can confidently answer "yes" to each of these:

Intervention Selection

- Can I identify whether a problem is a skill gap or a safety constraint?
- Do I know when to use few-shot examples vs programmatic hooks vs architectural changes?
- Can I apply the intervention hierarchy (descriptions → few-shots → prompt rules → hooks → architecture)?

Escalation Logic

- Can I list the three valid escalation triggers from memory?
- Can I identify invalid triggers (sentiment, confidence scores, discomfort, speculation)?
- Do I understand the difference between "policy gap" and "complex but within policy"?

Patterns

- Can I explain the evaluator-optimizer pattern and when it beats few-shot examples?
- Can I describe the case facts block pattern and why it's better than improving summarization prompts?
- Do I understand why multiple customer matches require clarification, not ranking algorithms?
- Can I explain why `stop_reason` is the loop control signal, not natural language parsing?

Tool Design

- Do I know what makes a good tool description (purpose, inputs, examples, edges, boundaries)?
- Can I identify keyword interference in system prompts?
- Do I understand parallel tool batching and when to use it?

Structured Errors

- Can I name the four error categories (transient, validation, business, permission) and the agent behavior each requires?
- Can I explain why generic error messages prevent intelligent recovery?

Handoffs and Hooks

- Can I list the four required fields in a structured handoff summary?
- Can I distinguish the three programmatic hook types: prerequisite gates, PostToolUse hooks, and pre-call interception?
- Do I know when to use pre-call interception vs prerequisite gates (parameter-based policy vs step ordering)?

Context Management

- Can I explain why verbose tool outputs should be trimmed and how this complements the case facts block?
- Do I understand the lost-in-the-middle effect and how to mitigate it with position-aware ordering?
- Can I describe the frustrated-but-resolvable customer handling pattern (acknowledge, offer, escalate only if reiterated)?

Pick the right level of intervention. Not every problem needs an architectural change; not every problem can be fixed with a better prompt. Knowing which is which is the central decision repeatedly tested.